Explaining a classifier's decisions

Noé Mastrorillo, Matthias Tortosa, Axel Pigeon, and Antoine Sinatti

University of Toulouse, France
{noe.mastrorillo, matthias.tortosa, axel.pigeon, antoine.sinatti}@utoulouse.fr

Abstract. As artificial intelligence and complex classifiers become increasingly prevalent in sensitive domains, the ability to understand and trust the decisions of these black-box models is essential. This document provides a comprehensive overview of current explainable AI (XAI) methods designed to interpret classifier outputs. The analysis categorizes these techniques into three primary families: logic-based abductive methods, game theory-based Shapley value approaches, and perturbation-based model-agnostic systems. First, logic-based methods attempt to formalize the classifier's reasoning to offer formal guarantees on the quality and rigor of the produced explanations. Second, Shapley value methods evaluate feature importance by assigning a contribution score to each input based on game theory principles. While these scores are easily interpretable for users, computing them can be computationally intractable and heavily dependent on characteristic functions and reference distributions. Finally, perturbation methods observe model behavior by generating perturbed input samples. These methods provide local explanations either through linear approximations or user-friendly IF-THEN rules. Ultimately, the document concludes that the choice of an explanation method depends on the desired balance between formal reliability, computational complexity, and user interpretability.

Keywords: Explainable AI · Classifiers · Black-box models · Feature Importance

Table of Contents

Explaining a classifier's decisions 1
Noé Mastrorillo, Matthias Tortosa, Axel Pigeon, and Antoine Sinatti
1 Abductive Explanation methods 2
1.1 AXp (Abductive Explanations) 3
1.2 CPI-Xp (Coverage-based Explanations) 3
1.3 Efficient and Rigorous Model-Agnostic Explanations 4
1.4 Leveraging Argumentation for Generating Robust Sample-based Explanations 5
2 Explanation methods based on Shapley values 6
2.1 Game theory preliminaries 6
2.2 Shapley values for feature explanation 7
2.3 Approximation 8
2.4 Different methods 9
2.5 Synthesis 10
3 Perturbation model-agnostic methods 10
3.1 LIME (Local Interpretable Model-Agnostic Explainer) 11
3.2 Anchors 13
3.3 Related 14
4 Conclusion 16

Introduction

For the last decades, Artificial Intelligence has become a powerful tool for companies but also for personal use, especially as home assistant. The famous ones are Large Language Models such as ChatGPT, Gemini, and others because we can naturally chat with them. The other famous type of models are classifiers and are still very used today. A classifier is a model (Random Forest, Neural Network, etc...) which sorts inputs into *classes*. For example, we can train a classifier to recognize cat images from dog ones or tumor scan from benign outgrowth scan. The natural question is then "How to trust the decisions of a classifier?" For non critical uses, we can simply test the model for several inputs to see if the model is able to recognize cats from dogs. But in sensitive domains, we may want to verify that the model effectively recognizes a cat instead of a wool ball or a cat tree in background. In this case, the model could effectively make an error and class a tumor scan in the benign scan class and then create a medical error.

When we use non-complex models such as small Random Forest, we can verify each decision of the model during the inference process. But for more complex ones like Neural Networks with billions of parameters, the human eye cannot do the same. That's why, researchers try to develop *explanation methods* which aims to provide a human comprehensible output about the black-box model decision

process for the user. The goal is to explain why the model took a certain decision and to figure out the origin of the potential errors.

This document details the currently-used models and systems for classifier explanation. We start by presenting a theoretical approach with Abductive Explanation methods. This section tries to formalize a classifier's reasoning to provide a formal guarantee of the produced explanation. Then, we present methods based on Shapley values that assign an importance value to each input feature of a decision, based on game theory principles. Finally, we present perturbation methods with LIME and Anchor systems. We present the theoretical aspects of these methods and their ameliorations.

1 Abductive Explanation methods

In this section, we present abductive explanation methods for classifiers, that provide *formal guarantees* on the quality of the produced explanations. We focus on two works: the algorithmic approach of **ignatiev_abduction-based_2019**, which introduces AXp's, and the theoretical extension of **cooper_abductive_2024**, which generalizes them to constrained settings via CXp's.

Remark 1 (Notation). We consider a classifier $\kappa : \mathbb{F} \to Cl$ where $\mathbb{F}$ is the feature space and Cl the set of classes. An instance $x \in \mathbb{F}$ is described by a set of literals (f_i, v). A partial assignment $E \subseteq x$ is a subset of literals of x. We write $E(y)$ to mean that instance y agrees with E on all features it defines.

1.1 AXp (Abductive Explanations)

ignatiev_abduction-based_2019 propose a constraint-agnostic framework to compute minimal explanations for any ML model. The core idea is to encode the classifier as a set of constraints $\mathcal{F}$ in a formal system (e.g. MILP or SMT), so that finding an explanation reduces to computing a *prime implicant* of $\mathcal{F}$.

> **Definition 11** *AXp*
>
> Let $x \in \mathbb{F}$ and $\kappa(x) = c$. A weak AXp of $\kappa(x)$ is a partial assignment $E \subseteq x$ such that:
>
> $$\forall y \in \mathbb{F},\ E(y) \Rightarrow \kappa(y) = \kappa(x)$$
>
> An AXp is a subset-minimal weak AXp. Intuitively, an AXp is the smallest set of feature values that is *sufficient* to guarantee the prediction, regardless of the values of the remaining features.

The paper distinguishes two types of minimality. A *subset-minimal* AXp contains no unnecessary literal and is computed in a linear number of oracle calls: each literal is removed in turn and kept only if its removal breaks the implication. A *cardinality-minimal* AXp is the shortest possible explanation, but its computation is Σ_2^P-hard and relies on an implicit hitting set enumeration. In

practice, subset-minimal AXp's are preferred as they are significantly cheaper to compute while remaining close in size to cardinality-minimal ones.

For neural networks with ReLU activations, the authors use a MILP encoding where Boolean variables $z_i \in \{0, 1\}$ capture the activation state of each neuron via indicator constraints ($z_i = 1 \rightarrow y_i \leq 0$). Experiments on UCI datasets and MNIST show that subset-minimal AXp's reduce instance size to 28–87% of the original features on average, and that the MILP oracle consistently outperforms SMT solvers. Compared to BDD-based explanations for Bayesian classifiers **shih_symbolic_2018**, AXp's on neural networks yield significantly more concise explanations (3–5 literals vs. 9).

The main limitation of AXp's is that they implicitly assume *independence* between features. When dependencies exist (e.g. a pregnant person is necessarily a woman), AXp's can produce superfluous or redundant explanations, and their number can grow exponentially [**cooper_abductive_2024**].

1.2 CPI-Xp (Coverage-based Explanations)

cooper_abductive_2024 address this limitation by introducing coverage-based explanations (CPI-xp). The key notion is the *coverage* of a partial assignment E, defined as the set of feasible instances it explains:

$$\mathrm{cov}(E) = \{x \in \mathbb{F}[\mathcal{C}] \mid E(x)\} \tag{1}$$

where $\mathbb{F}[\mathcal{C}]$ is the set of instances satisfying the constraints $\mathcal{C}$. An explanation E' *subsumes* E if $\mathrm{cov}(E) \subseteq \mathrm{cov}(E')$, meaning E' applies to at least as many instances as E.

> **Definition 12** *CPI-Xp*
> A coverage-based PI-explanation (CPI-Xp) of $\kappa(x)$ is a partial assignment E such that:
>
> (i) $E(x)$
> (ii) $\forall y \in \mathbb{F}[\mathcal{C}],\ E(y) \Rightarrow \kappa(y) = \kappa(x)$
> (iii) no other valid explanation E' strictly subsumes E in $\mathbb{F}[\mathcal{C}]$

By selecting only explanations of maximal coverage, CPI-Xp automatically discard redundant and superfluous ones. The authors further refine this notion: a *minimal* mCPI-Xp is subset-minimal among CPI-Xp's, and a *preferred* pCPI-Xp keeps only one representative among equivalent ones (same coverage). The pCPI-Xp is the only notion satisfying all seven desirable formal properties defined in the paper (success, non-triviality, irreducibility, coherence, consistency, independence, non-equivalence).

However, computing CPI-XP over the full feature space is Π_2^P-complete, a higher complexity class than AXp's (FP^{NP}). To make the approach tractable for black-box classifiers, the authors propose *dataset-based* versions (d-CPI-Xp)

where coverage is computed over a sample $T \subseteq \mathbb{F}[\mathcal{C}]$ only. This reduces complexity to polynomial time $O(m^2n^2)$, but at the cost of explanations that are only valid for instances in T, losing the global coherence property.

1.3 Efficient and Rigorous Model-Agnostic Explanations

Logic-based XAI addresses the lack of rigor of sub-symbolic methods but has two major drawbacks, the need to access the structure of the model and the scalability problems. The paper "Efficient and Rigorous Model-Agnostic Explanations" [**marques-silva_efficient_2025**] tries to solve both problems by combining both. To do that it will get the sbAXp, the sample based abductive explanation, rigorously :

$$\mathbb{S} \subseteq \mathbb{F}, \forall (x \in \mathbb{S}). \left(\bigwedge_{i \in \mathcal{X}} (x_i = v_i) \right) \rightarrow (\kappa(x) = \kappa(v)) \tag{2}$$

This XAI uses weak contrastive explanation (sbWCXp) :

$$\mathbb{S} \subseteq \mathbb{F}, \exists (x \in \mathbb{S}). \left(\bigwedge_{i \in \mathcal{F} \setminus \mathcal{X}} (x_i = v_i) \right) \wedge (\kappa(x) \neq \kappa(v)) \tag{3}$$

which is a subset of features that suffice to change the prediction if their value is changed.

A Contrastive Explanation (CXp) is a subset-minimal WCXp, whereas an AXp answers a "Why?" question, a contrastive explanation answers a "Why not?" question. This XAI uses the duality of AXp and CXp explanations, "In general, each abductive explanation is a minimal hitting set of the set of contrastive explanations, and vice-versa" [**marques-silva_efficient_2025**], meaning, from sbCXp we can get the sbAXp.

The algorithm to go from the subset of all sbWCXp to one sbAXp is quite simple: compare each sbWCXp against all others and keep the ones with no subsets to get all sbCXps. **cooper_abductive_2024** work proposed an algorithm to get an sbAXp from all sbCXps but **marques-silva_efficient_2025** proposes an asymptotically more efficient solution, that runs in $O(mn)$ time instead of $O(m^2n)$.

Finding one sbAXp can be done in polynomial time but finding one smallest sbAXp is much harder and unlikely to be solved in polynomial time. Finding one smallest sbAXp can be done by encoding the problem to propositional maximum satisfiability (MaxSAT). With this they are able to find one smallest sbAXp in a similar time as finding one sbAXp.

1.4 Leveraging Argumentation for Generating Robust Sample-based Explanations

This sample-based XAI uses arguments [**amgoud_leveraging_2023**], which is a plausible sbAXp. To get the final sbAXps, this XAI uses attacks: if two

arguments contradict each other, then both will be compared and only one (if any) will be kept, this is repeated until there is no contradiction.

Each argument is composed of two parts, first the support, which the condition of the argument, when does it apply, second is the conclusion, what class does the argument predict. A contradiction is when the union of each support is not null and the conclusion different. All attacks are treated at the same time and depending on the selection function will give a set of stable extensions. A stable extension is a set of arguments without contradiction. The choosing of one of the stable extensions is done with an inference rule.

There are multiple selection functions : Max which gives all possible stable extensions. Card which only keep the ones with the highest number of arguments. $Incl_i$ et $Card_i$ which give the ones that cover the most instances in the datasets. And $Incl_c$ et $Card_c$ which give the ones that cover the highest number of classes.

To choose between all stable extensions, there are two inference rules possible : the universal inference that will only keep arguments that are in all selected stable extensions and the existential inference that will give all arguments in selected stable extensions.

- **Universal inference:** $\Sigma \vdash^{\forall} a$ iff $\forall \mathcal{E} \in \Sigma$, $a \in \mathcal{E}$.
- **Existential inference:** $\Sigma \vdash^{\exists} a$ iff $\exists \mathcal{E} \in \Sigma$ such that $a \in \mathcal{E}$.

Where $\mathcal{E}$ is a stable extension, a an argument and Σ all selected stable extensions.

With this method both success, finding an sbAXp and coherence, having an sbAXp with no contradiction is not guaranteed. Coherence is guaranteed with the universal inference rule while Coherence is often broken with the existential inference rule, thus prioritize universal inference.

2 Explanation methods based on Shapley values

Some of the most widely used methods for local features importance explanation are based on the Shapley values from game theory. These methods are so popular because they give a contribution score for each feature in the decision that is easily understood by the user. This contribution score has a sign and a magnitude, so that a positive score means that the feature contributes positively to the decision and the opposite for a negative score. This results in straightforward visualizations of local features importance, as displayed in Figure 1 that shows Shapley-based contribution scores of features for a prediction of a naive Bayes classification model on an instance of the famous Titanic dataset. The goal here is to predict if the person represented by x would have been able to survive from the Titanic. The model predicted the class *yes*, and the scores tell us that the main contribuing feature is the *Sex*: because the person is a female, she has good chances to survive. We can also see that the *Class* of the person works slightly against its survival, as well as the fact that its an *adult*.

While these methods are amongst the most popular, that doesn't mean that they are superior, as we will see the several shortcomings later.

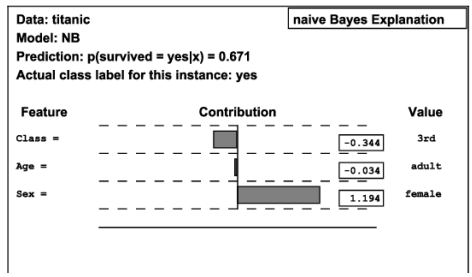

Fig. 1: Feature importance attributions computed with Shapley values for the decision of a naive Bayes model on an instance of the Titanic dataset. From **strumbelj_efficient_2010**.

2.1 Game theory preliminaries

In game theory, Shapley values provide a mathematically fair and unique method to attribute the payoff of a cooperative game to the players. Formally, a coalitional game $(\mathcal{F}, v)$ is composed of a set of n players $\mathcal{F} = \{1, \ldots, n\}$, and a characteristic function $v : \mathcal{P}(\mathcal{F}) \to \mathbb{R}$ that represents the payoff for a subset of players $\mathcal{S} \subset \mathcal{F}$ (a coalition). Then, the Shapley value for a single player $i \in \mathcal{F}$, noted $\varphi_i(v)$ is defined by **merrick_explanation_2020** as follows:

> **Definition 21** *Shapley value*
>
> $$\varphi_i(v) = \frac{1}{n} \sum_{\mathcal{S} \subset \mathcal{F} \setminus \{i\}} \binom{n-1}{|\mathcal{S}|}^{-1} \big(v(\mathcal{S} \cup \{i\}) - v(\mathcal{S})\big) \tag{4}$$

Therefore, the Shapley value for a player i is computed by going through all the coalitions of players not containing i and looking at the payoff difference with and without the player i (with a normalization factor).

The Shapley value of a coalitional game $(\mathcal{F}, v)$ is the only solution that respects the four following powerful axioms, as described by **fryer_shapley_2021**:

- *Efficiency*: The worth of the coalition containing all players is distributed losslessly among the players :

$$\sum_{i \in \mathcal{F}} \varphi_i(v) = v(\mathcal{F}) - v(\emptyset) \tag{5}$$

- *Null Player*: If a player contributes nothing to any coalition, then its Shapley value is zero :

$$\forall i \in \mathcal{F} \quad \big[\forall \mathcal{S} \subset \mathcal{F} \setminus \{i\} \quad v(\mathcal{S} \cup \{i\}) = v(\mathcal{S})\big] \Longrightarrow \varphi_i(v) = 0 \tag{6}$$

- *Symmetry*: Any two players that play equal roles amongst all coalitions receive the same Shapley value :

$$\forall i, j \in \mathcal{F} \quad \big[\forall \mathcal{S} \subset \mathcal{F} \setminus \{i, j\} \quad v(\mathcal{S} \cup \{i\}) = v(\mathcal{S} \cup \{j\})\big] \Longrightarrow \varphi_i(v) = \varphi_j(v) \tag{7}$$

– *Additivity*: Let v_1 and v_2 be two characteristic funtions, the Shapley value respects the additivity of functions :

$$\varphi(v_1 + v_2) = \varphi(v_1) + \varphi(v_2) \tag{8}$$

2.2 Shapley values for feature explanation

In ML, we treat each feature as a player, and the Shapley values allocate the predicted value of a specific instance fairly amongst all the features. Thus, the Shapley values represent the contribution (positive or negative) of each feature in the decision to explain.

Formally, let $x \in \mathbb{F}$ be the instance to explain where $\mathbb{F} = \mathbb{F}_1 \times \cdots \times \mathbb{F}_n$ is the input space of our model (each $\mathbb{F}_i$ is the space of the feature i), and $f(x)$ is the predicted value of our model for a specific class. We consider the coalitional game represented by $\mathcal{F} = \{1, \ldots, n\}$ the set of numbered features that will be our players, and the characteristic function $v : \mathcal{P}(\mathcal{F}) \longrightarrow \mathbb{R}$ defined as follows:

$$\forall \mathcal{S} \subset \mathcal{F} \quad v(\mathcal{S}) = \mathbb{E}[f(X)|X_{\mathcal{S}} = x_{\mathcal{S}}] \tag{9}$$

where $X_{\mathcal{S}} = x_{\mathcal{S}}$ means that instances X and x share the same values for the features in $\mathcal{S}$. The characteristic function is the mean value of the predictions on inputs that share the same values for the features in $\mathcal{S}$.

Finally, we can define the Shapley value of a feature i in the wish of explaining the prediction of an input x as follows:

Definition 22 *Shapley value for a feature i*

$$\varphi_i = \frac{1}{n} \sum_{\mathcal{S} \in \mathcal{F} \setminus \{i\}} \binom{n-1}{|\mathcal{S}|}^{-1} \left(\mathbb{E}[f(X)|X_{\mathcal{S} \setminus \{i\}} = x_{\mathcal{S} \setminus \{i\}}] - \mathbb{E}[f(X)|X_{\mathcal{S}} = x_{\mathcal{S}}] \right) \tag{10}$$

Remark 2. Every Shapley value computed will be based on the choice of the distribution of the reference instances X, and we will see later how it matters. Moreover, we can deduce from the above definition 22 and the efficiency axiom 5 the following property:

$$\sum_{i \in \mathcal{F}} \varphi_i(v) = \mathbb{E}[f(X)|X_{\mathcal{F}} = x_{\mathcal{F}}] - \mathbb{E}[f(X)|X_{\emptyset} = x_{\emptyset}] = f(x) - \mathbb{E}[f(X)] \tag{11}$$

Therefore, the sum of the Shapley values of all features equals to the predicted value minus a *reference contribution* that is the mean of all predictions (for inputs based on a specific distribution).

2.3 Approximation

As you may have noticed, computing exact Shapley values has an exponential complexity in terms of the number of features. Indeed, it requires exploring all the subcoalitions $\mathcal{S}$ of $\mathcal{F}\backslash\{i\}$. In practice, this problem becomes intractable when playing with hundreds of features (see **strumbelj_efficient_2010**). Moreover, the calculation of the characteristic function $v(\mathcal{S})$ is also intractable as it requires exploring the input space $\mathbb{F}$.

Both issues are solved notably by **merrick_explanation_2020** by using sampling methods. For the former, sub-coalitions $\mathcal{S} \subset \mathcal{F}\backslash\{i\}$ are sampled by taking all the features preceding i in random orderings O_j of $\mathcal{F}$: we define $\mathcal{S}_j = Pre_i(O_j)$. For the latter, ehe reference instances X are sampled following the chosen distribution.

Remark 3. These two solutions allow us to build confidence intervals over the approximated Shapley values, and we can control the approximation error from the exact Shapley values by varying the number of samples drawn (if we want a confidence interval closer to the exact value, we have to draw more samples). Each feature contribution can now be represented as an interval. This is particularly interesting because this gives us more information whether an approximation of the contribution is relevant or not. If a Shapley value is approximated to be close to zero, its confidence interval may take values from negatives to positives values, meaning that we cannot deduce any valuable information about whether this feature contributes positively or negatively in the decision. Remember that if we could compute the exact Shapley value, this case would not exist.

2.4 Different methods

Several explanation methods based on Shapley values have been proposed through the past years. What is described above is the original SHAP (SHapley Additive exPlanations) framework, that serves as the groundwork for newer methods. Various papers pinpointed the shortcomings of SHAP (see **letoffe_towards_2025** and **fryer_shapley_2021**), and mainly the fact that it can attribute more importance to features having less influence (or even no influence at all) in the decision.

On the one hand, **letoffe_towards_2025** claim that this issue can be solely attributed to the characteristic function used, and the authors propose new characteristic functions based on abductive and contrastive explanations (AXps and CXps). They introduce a similarity predicate σ defined below and one example of characteristic function v_a associated :

$$\sigma(y) = \begin{cases} 1 & \text{if } f(y) \approx f(x) \\ 0 & \text{else} \end{cases} \qquad v_a(\mathcal{S}) = \begin{cases} 1 & \text{if } \mathbb{E}[\sigma(X)|X_{\mathcal{S}} = x_{\mathcal{S}}] = 1 \\ 0 & \text{else} \end{cases} \tag{12}$$

Concretely, $v_a(\mathcal{S}) = 1$ if all the instances sharing the same values as x on the features in $\mathcal{S}$ get predicted a similar value (same class in the case of classification). Therefore, $v_a(\mathcal{S})$ is equivalent as knowing if $\mathcal{S}$ is a weak abductive

explanation (wAXp) of x. v_a and the other proposed characteristic functions respect two important properties: *strong value independance* (mapping the input space into another one by a surjection doesn't change the value of the characteristic function) and *feature irrelevancy* (a feature that doesn't influence the prediction gets attributed a Shapley value of 0). These properties are desirable to get around the previously mentionned issues.

On the other hand, **merrick_explanation_2020** pinpoint the importance of the choice of the reference distribution, noted $\mathcal{D}^{ref}$, used in the characteristic function to compute $\mathbb{E}[f(X)|X_{\mathcal{S}} = x_{\mathcal{S}}]$. Indeed, we could decide that X follows the input distribution (distribution of the dataset) $\mathcal{D}^{inp}$ (used in SHAP and KernelSHAP), the joint-marginal distribution of features $\mathcal{D}^{J.M.}$ (used in QII), or even a uniform distribution over the input space $\mathcal{D}^{unif}$. While chosing $\mathcal{D}^{inp}$ takes into account only instances present in the dataset and can suffer from the proeminance of some feature values, chosing $\mathcal{D}^{J.M.}$ assumes independance between all the features, and like $\mathcal{D}^{unif}$, can sample unrealistic instances that will have an impact in the computed attributions. These choices are directly reflected in the computed SHAP scores, revealing why different methods can give very different SHAP scores. Therefore, the paper proposes the Formulate-Approximate-Explain framework when facing this choice, saying that we should firstly formulate a contrastive question related to the classification task to pin down the desired reference distribution. The choice in distribution lies in the question of "Why does this instance received this score *in contrast of* these other references".

Finally, there exists model-specific SHAP-based methods for explaining decisions of classifiers with known designs. For instance, **campbell_exact_2022** present Eject, intended to give more model-related explanations while solving shortcomings of previous TreeSHAP method that computes SHAP scores on Tree-based models (classification trees and trees ensembles models like random forests). Eject computes the utility of a coalition of features for a specific instance by going down the decision branch and *ejecting* (returning the predicted value of the previously visited node) when the branch uses a feature not present in the coalition. This notably ensures the *local dummy player* property: a feature not used in the prediction gets a score of 0. Also, Eject method allows to compute exact Shapley values rather than approximations. On the other hand, CNN SHAP proposed by **zhao_shap_2021** aims to compute SHAP scores on convolutive neural networks (more precisely CNN text classification models), and thus on non-structured data. CNN SHAP firstly let the convolutive layers of the model extract the semantic features, and compute the SHAP scores on these extracted features, while keeping in memory the part of the input corresponding to each feature. Then, there's a deduplication step for features having similar semantic meanings and a merging step for features whose corresponding input parts overlap. Finally, the computed SHAP scores can directly be attributed to the input areas.

2.5 Synthesis

In summary, Shapley-based methods give local explanation of feature contributions. While some model-specific methods ensure the tractability of exact computation of Shapley values, this is an exponential complexity problem in terms of the number of feature subsets, solved by sampling-based approximation methods. Different methods have been developped to surpass the shortcomings of the original SHAP framework, by modifying the characteristic function used in the computation of the Shapley values, or the distribution of sampled reference instances that plays a major role in the liability of the contribution scores.

3 Perturbation model-agnostic methods

In this section, we present a set of perturbation methods used in black-box models explanation. A perturbation method can be defined by the generation of perturbed samples (ex: noised features in model representation or perturbed input tensor) at the input of the black-box model. The objective is to compare the model behavior (the prediction) between a set of different perturbed inputs and the original one. Then, we can determine which one is more important for the prediction. It allows the explanation model to detail why the model took a specific decision. The following methods are of this type. The first one, LIME (Local Interpretable Model-Agnostic Explanation) focus on a linear approximation of local model behavior. Secondly, the Anchors method provides an amelioration of LIME by introducing the concept of IF-THEN rules to provide a better user-experience. Finally, we will briefly talk about add-ons of those methods in specific cases.

Remark 4 (Notation). Let's introduce some notations to identify which objects we will talk about. We denote with f a black-box model we want to explain, then $f : X \longrightarrow Y$ where X is the set of instances $x \in X$ and Y the different possible predictions $f(x)$. In this section, we will talk about linear explanation model, denoted g. To identify which input representation we use, we denote by x' the interpretable representation of an input x in the black-box model f. Then, if we construct a set of perturbed interpreted samples $\mathcal{Z}$ from x', this set contains perturbed version of x' called z'. Finally, $\Omega(g)$ express the complexity of the model g, generally defined by the number of non-zeros weights in g.

3.1 LIME (Local Interpretable Model-Agnostic Explainer)

LIME founded the principe of perturbation method for classifiers explanation [**ribeiro_why_2016**]. It provides a local explanation of a black-box model behavior $f : \mathbb{R}^d \longrightarrow \mathbb{R}$ by training a simple explanation model g over perturbed samples. This method is *model-agnostic* because it doesn't depend on the black-box model architecture. f is considered as an oracle used to train g. The resulting explanation is the set of the weights of g which give a local importance of each f input features.

Formally, for an original interpretable representation $x' \in \{0,1\}^{d'}$ of a sample $x \in \mathbb{R}^d$, it generates a set of perturbed samples z' which are drawn by selecting uniformly some x' non-zero components. This random sampling is weighted by π_x which defines a locality around x' denoted $\mathcal{Z}$. In practice, π_x is an exponential kernel applied to a distance D. To find the optimal model g, we can define a loss function $\mathcal{L}(f, g, \pi_x)$ telling how unfaithful g is in approximating f on $\mathcal{Z}$. The objective is then to minimize $\mathcal{L}$ and having $\Omega(g)$ low enough for human comprehension. The LIME explanation is then produced by:

$$\xi(x) = \arg\min_{g \in G} \mathcal{L}(f, g, \pi_x) + \Omega(g) \quad (13)$$

with G the set of available explanation models [**ribeiro_why_2016**]. Using $\mathcal{L}$ and $\Omega(g)$ to find the best model g ensures *interpretability* and *local fidelity* on the black-box model explanation.

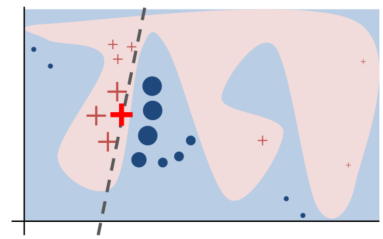

Fig. 2: **LIME linear local approximation. Black-box model f classes is represented in blue/gray. The red cross represents an explained instance and the others symbols are the perturbation samples generated from the original. The dotted line represents the local approximation of g.**

Finally, the local explanation of the model is provided by the weights w_g trained with $g(z') = w_g \cdot z'$.

As stated at beginning, LIME provides a *local* explanation of f. To ensure a global explanation, the paper proposes a Sub-modular Picks LIME (SP-LIME) which selects a set of local and non-redundant LIME explanations to be given to the user [**ribeiro_why_2016**]. To choose those explanations, the paper introduces the concept of coverage of a set of instances which defines the importance of those instances.

Definition 31 *Coverage*

Let g be an explanation model and $g_i = \xi(x_i)$ the explanation of the instance x_i, we construct the $|X| \times d'$ explanation matrix $\mathcal{W}$ with $\mathcal{W}_{i,j} = |w_{g_{i,j}}|$. Then, I_j is the j-column of W and denotes the global importance of that component in the explanation space. Then, the coverage of a set of instances V is defined as the total importance of the features that appear at least one instance in V:

$$c(V, \mathcal{W}, I) := \sum_{j=1}^{d'} \mathbb{1}_{\{\exists i \in V, \mathcal{W}_{i,j} > 0\}} I_j \quad (14)$$

Finally, from a maximum number of global explanations B set by the user, the pick problem is defined by:

$$Pick(\mathcal{W}, I) = \underset{V, |V| \leqslant B}{\arg\max}\, c(V, \mathcal{W}, I) \tag{15}$$

We want to select the highest coverage and non-redundant explanations to provide a global one.

Unfortunately, this system has limits. First, in practice, the pick problem is intractable, so a greedy algorithm is used to compute it. Secondly, LIME algorithm doesn't define clear boundaries of the explanation instances. Indeed, all black-box models cannot be locally approximated by a linear one [**ribeiro_nothing_2016**]. Finally, the explanation results depend on the LIME parameters such as B, the distance metric D and the kernel π_x. Changing one of them can change the interpretation and undermining the reliability of this method [**wilking_fooling_2022**].

3.2 Anchors

To address the lack of clear coverage boundaries on LIME, **ribeiro_nothing_2016** introduced anchor-LIME (aLIME). This LIME evolution uses IF-THEN rules to 'anchor' a prediction. This transition to linear importance to rule-based invariance laid the foundation of the Anchors method. Thus, they are defined by short and disjoint rules which are preferred by users for explanation [**ribeiro_anchors_2018**]. This method can be applied on different set of domains (tabular, text, images, etc...) for different tasks (classification, prediction, text generation, etc...).

Definition 32 *Anchors*

Formally, Anchors are built on a set of predicates $\{a_i\}$ called a "rules". We can also see it as a function of instances $x \in X$:

$$A : \begin{cases} X \longrightarrow \{0, 1\} \\ x \longmapsto \begin{cases} 1 \text{ if all rules are true for } x \\ 0 \text{ otherwise} \end{cases} \end{cases}$$

to link a set of rules A on a prediction $f(x)$ (where $f : X \longrightarrow Y$ is the black-box model). Finally, A is an Anchor of x if $A(x) = 1$ and A is a sufficient condition for the prediction $y = f(x)$:

$$\mathbb{E}_{\mathcal{D}(z|A)}[1_{f(x)=f(z)}] \geqslant \tau \tag{16}$$

with τ the desired level of precision of the Anchor on x [**ribeiro_anchors_2018**]. Here, $\mathcal{D}(x \mid A)$ represents the distribution of neighbors of x which satisfy the Anchor A.

Computing Anchors is the difficult part of the process. This is a combinatorial optimization problem if we decide to choose anchors which satisfy a precision

constraint with high probability:

$$\mathbb{P}(prec(A) \geqslant \tau) \geqslant 1 - \delta$$

with $prec(A) = \mathbb{E}_{\mathcal{D}(z|A)}(1_{f(x)=f(z)})$ called the Anchors "precision". Among those anchors, we choose the largest coverage ones, where coverage is :

$$cov(A) = \mathbb{E}_{\mathcal{D}(z)}(A(z)) \tag{17}$$

Finally, the optimization problem can be described by :

$$\max_{\mathbb{P}(prec(A)\geqslant\tau)\geqslant 1-\delta} cov(A) \tag{18}$$

Because the number of all possible Anchor is exponential, we cannot perform an exhaustive search. That's why the article introduced the Bottom-Up approach:

1. *Initialization*: The anchor A is initialized as an empty rule (applying to all instances).
2. *Candidate Generation*: In each iteration, a set of candidate rules $\mathcal{A}$ is generated by extending A with one additional feature predicate $\{a_i\}$: $\mathcal{A} = \{A \wedge a_i, A \wedge a_{i+1}, A \wedge a_{i+2}, ...\}$
3. *Selection via Multi-Armed Bandit (MAB)*: To identify the best candidate without an exhaustive number of calls to f, the problem is formulated as a pure-exploration multi-armed bandit problem. The *KL-LUCB* algorithm is used to find the rule with the highest precision using a minimal set of samples [**ribeiro_anchors_2018**].
4. *Termination*: The process terminates when a rule meets the statistical precision constraint: $\mathbb{P}(prec(A) \geqslant \tau) \geqslant 1 - \delta$.

To address the limitations of a purely greedy search, the algorithm extends the construction to a *Beam Search* of size B. It maintains a set of B current candidates instead of a single one. Among the multiple anchors identified that satisfy the precision threshold τ, the algorithm outputs the one with the highest coverage $cov(A)$, effectively optimizing the original problem.

To provide a global understanding of the model, the article adapts a *Submodular Pick* (SP) procedure. The goal is to select a set of K anchors that are non-redundant and cover as many instances in the validation set similar to the approach used in LIME [**ribeiro_anchors_2018**, **ribeiro_why_2016**].

3.3 Related

Let's talk about related topics about Anchors and LIME and specifically about improvements provided in specific cases.

The first one is an amelioration of LIME weights for an instance. The related work is motivated by the fact that LIME system define the importance of a feature only by the distance between its original representation and the noised

one in the model representation. With ILIME, the paper proposes a new method which consider the black-box loss function value between the two instances, this defines the *influence* of a feature. Formally, it is defined as scalar product between original weight of the prediction on the perturbed feature and the loss function gradient evaluated at this point. The final "importance" value of a feature on the model's prediction is defined as the product of its influence and its original LIME weight. Thus, a synthetic instance will have an important weight if its representation is near to the original feature one, and if it is able to modify the model prediction. To provide a global explanation of the model behavior, ILIME uses a set of local explanations and defines a Dendogram with hierarchical clustering on explanations (weights). It differs from LIME where the user needed to define the number of global explanation we wanted.

Secondly, **ribeiro_nothing_2016** presents the evolution of LIME system by adding Anchors to clearly define LIME coverage boundaries. Indeed, by construction LIME perturbs the output representation x' of a black-box model f by selecting random x' components according to a distribution π_x. However, this π_x doesn't define a clear zone around x' where the system select the perturbed inputs to give to the models, some can be very close and the other very far. So that human cannot really understand which feature is more important than another. That's why, aLIME is an IF-THEN rules-based system that "anchors" a prediction features such that changing the others don't change the black-box model output. By this approach, precision (defined as how accurate humans are in those predictions) is needed but a too large set of rules only specific to a single instance is not useful for model understandability. Then the combinatorial aLIME problem is to find the shortest Anchor with high precision.

As presented in the previous section, SHAP-Scores in an intractable problem. Fortunately, **arenas_tractability_2021** presents an efficient algorithm to compute SHAP-scores within specific classification models. The SHAP-score is a prominent explanation method for classifiers that assigns a numerical value to each input feature based on its contribution to the decision process. In general, computing SHAP-scores is computationally intractable due to the model's complexity, often being #P-hard. However, this paper focuses on *Boolean Classifiers*, which return either 1 (accept) or 0 (reject) for a given input. These models are represented as *deterministic* and *decomposable* Boolean circuits, a class that encompasses models like Bayesian Classifiers, Random Forests, and Decision Trees. In this framework, the authors provide a polynomial-time algorithm to compute SHAP-scores over tractable Boolean circuits, initially assuming a uniform distribution over inputs. Both the determinism and decomposability properties are strictly necessary for this tractability:

- Determinism ensures that for every OR gate, no two inputs can be true at the same time for any given assignment.
- Decomposability requires that for every AND gate, the sets of features in its sub-circuits are pairwise disjoint (they share no variables).

The core contribution of the paper is the reduction of the SHAP-score computation to a #SAT problem (model counting). The authors then define a

polynomial-time algorithm to solve this specific #SAT problem using a *bottom-up induction* process.

Finally, we can discuss an implementation of SHAP-Scores and Anchors in financial fraud detection. Indeed, **thanathamathee_shap-instance_2024** presents a real-case application of SHAP-values and Anchors in fraud detection systems. While traditional statistical methods are common, they struggle with the complex nature of modern fraud. The authors enhance the XGBoost model by using a novel SHAP-instance weighting method to prioritize learning from the most informative samples. To solve the massive class imbalance in the Thailand Stock Exchange dataset, they combine RENN and SMOTE techniques before optimizing the model with Optuna and Hyperband. The resulting model effectively differentiates between non-fraudulent, fraudulent, and "grey area" behaviors. By reaching a perfect recall (1.000) for fraudulent cases, this approach sets a new standard for accuracy and trust. Furthermore, Anchors are used to translate "black-box" predictions into easy-to-understand IF-THEN rules, providing clear insights for risk management. However, integrating these SHAP and Anchor systems increases the computational load and total processing time.

In summary, LIME defined the principle of perturbation explainer by learning weights which denotes an instance importance in the black-box decision process. Then, Anchors enhanced this model by adding IF-THEN rules which allows to give clear instances coverage to the user. Nevertheless, this precision is difficult to reach because of the exponential complexity fo this type of computation. In practice, Anchors and LIME are used in specific domains where explainability is necessary to trust the model.

4 Conclusion

We've presented three major families of local decision explainers for classifiers : logic-based methods with AXps and CXps, Shapley values-based methods with SHAP and its derivatives, and perturbation based methods with LIME and Anchor. The choice in the used method depends on what type of explanation is expected. Logic-based methods give sets of the most relevant features that made the decision and provide formal guarantees on the quality of the explanations. SHAP-based methods attribute a contribution score to each feature in a decision that is easily interpretable for non-expert users, but may be less reliable. Similarly, LIME gives the linear weight attributed to each feature in a local submodel around the decision, so that each weigth represent the importance of each feature in the decision. And finally, Anchor provides user-friendly IF-THEN rules to trace down the decision.

Table 1 shows a comparison of all the previously mentioned methods for local explanation of classifier decisions. In the model type column, *Black-box* means that the method can be used to explain the decisions of any model, without knowledge of its internal workings. While most of the methods can be used on black box models, some are model-specific (they work on a specific type of models) which is restrictive but can have some benefits in other columns

(in the complexity for instance). The algorithmic complexity of each method is expressed relative to the number of features of the model, and some may have an appoximation algorithm that have a better complexity. Finally, the reliability may seem vague, but it's a way to rank the methods in terms of the trustability we can put in the explanations they provide.

Table 1: Comparative analysis of XAI methods (Perturbation vs. Abductive approaches).

Method	**Model type**	**Complexity**	**Approx.**	**Reliability**
sbAXp	Black-box	Polynomial		+
Argument	Black-box	Polynomial		++
CPI-Xp	Black-box	Π_2^P-complete	Polynomial	+
SHAP	Black-box	Exponential	Polynomial	-
SHAP with v_a	Black-box	Exponential	Polynomial	+
Eject	Tree ensembles	Polynomial		+
CNN SHAP	Convolutive Neural Networks	Exponential	Polynomial	-
LIME	Black-box	Polynomial		-
Anchors	Black-box	Exponential		+

While a lot of work on explainable AI has been published in the past years, validating models still represents a great challenge. Lots of larger AI systems emerge regularly but few people actually give importance to the reliability of their systems, which is mandatory in safety-critical contexts. We hope that we were able to raise people's awarness on the interpretability of AI models with initial lines of inquiry, and that this domain will continue to grow.